File handling

PAGE NO.....
DATE / /

→ allows you to read from and write to files on your computer.

1. Opening a file

file_object = open (filename, mode).

- filename: The name of the file (including the path if necessary)
- mode: The mode in which the file is opened. (e.g. read, write, append).

common Modes

mode	Descriptions.
'r'	Read mode
'w'	write mode.
'a'	append mode.
'b'	binary mode.
'x'	Exclusive creation mode. Fails if the file already exists.
'+'	update mode (e.g. 'r+' for reading and writing).

Eg.

file = open ("example.txt", "r").

PAGE NO.....
DATE / /

2) Reading from a file.

(a) read().

```
file = open ("example.txt", "r").
    content = file.read()
    print (content).
    file.close()
```

→ Reads the entire file as a string.

(b) readline().

```
file = open ("example.txt", "r").
    line = file.readline()
    print (line).
    file.close()
```

→ Reads one line at a time.

(c) readlines()

```
file = open ("example.txt", "r")
    line = file.readline()
    print (line)
    file.close()
```

→ Reads all lines into a list.

(d) Iterating over lines.

```
file = open ('example.txt', "r").
for line in file:
        print (line.strip()). # strip() removes
file.close().                   newline characters
```

PAGE NO.....
DATE / /

3) Writing to a file

(a) write():

```
file = open("example.txt", "w")
file.write("Hello, World!\n")
file.close()
```

→ writes a strings to the file.

(b) writelines:

```
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]
file = open("example.txt", "w").
file.writelines(lines).
file.close()
```

→ writes a list of strings to the file.

4) Using with for file handling

→ The with statement ensures that the file is automatically closed when the block is exited, even if an exception occurs.

```
with open("example.txt", "r") as file
    content = file.read()
    print(content).
# file is automatically closed here.
```

PAGE NO.....
DATE / /

5) File Handling with Binary File.

```
with open ("image.png", "rb") as file
    data = file.read()

with open ("copy.png", "wb") as file
    file.write(data)
```

6) File handeing with JSON.

(a) Reading JSON

```
import json
with open ("data.json", "r") as file:
    data = json.load(file)
    # Load JSON data into a Python
      dictionary
    print(data)
```

(b) Writing JSON.

```
Import json
data = {"name": "Alice", "age": as}
with open ("data.json", "w"} as file:
    json.dump(data, file)
    # write Python dictionary
      to JSON file.
```

PAGE NO.....
DATE / /

7) File Handling with CSV.

(a) Reading csv.

```
import csv
with open('data.csv', 'r') as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

(b) Writing csv

```
import csv
data = [['Name', 'Age'], ['Alice', 25], ['Bob', 30]]
with open("data.csv", "w") as file:
    writer = csv.writer(file)
    writer.writerows(data)
```

8) File Handling with Pickle.

-> The pickle module allows you to serialize and deserialize python objects (e.g. lists, dictionaries) to and from files.

(a) Writing to a Pickle file

```
import pickle
data = {"name": "Alice", "age": 25}
with open("data.pkl", "wb") as file:
    pickle.dump(data, file)
```

(b) Reading from a Pickle file

```
import pickle
with open("data.pkl", "rb") as file:
    data = pickle.load(file)
```

PAGE NO.....
DATE / /

print (data).